

**10-2. Learning C:** You can use the `replace()` method to replace any word in a string with a different word. Here's a quick example showing how to replace 'dog' with 'cat' in a sentence:

---

```
>>> message = "I really like dogs."
>>> message.replace('dog', 'cat')
'I really like cats.'
```

---

Read in each line from the file you just created, *learning\_python.txt*, and replace the word *Python* with the name of another language, such as *C*. Print each modified line to the screen.

**10-3. Simpler Code:** The program *file\_reader.py* in this section uses a temporary variable, `lines`, to show how `splitlines()` works. You can skip the temporary variable and loop directly over the list that `splitlines()` returns:

---

```
for line in contents.splitlines():
```

---

Remove the temporary variable from each of the programs in this section, to make them more concise.

## Writing to a File

One of the simplest ways to save data is to write it to a file. When you write text to a file, the output will still be available after you close the terminal containing your program's output. You can examine output after a program finishes running, and you can share the output files with others as well. You can also write programs that read the text back into memory and work with it again later.

### Writing a Single Line

Once you have a path defined, you can write to a file using the `write_text()` method. To see how this works, let's write a simple message and store it in a file instead of printing it to the screen:

---

```
write from pathlib import Path
_message.py
path = Path('programming.txt')
path.write_text("I love programming.")
```

---

The `write_text()` method takes a single argument: the string that you want to write to the file. This program has no terminal output, but if you open the file *programming.txt*, you'll see one line:

---

```
programming.txt I love programming.
```

---